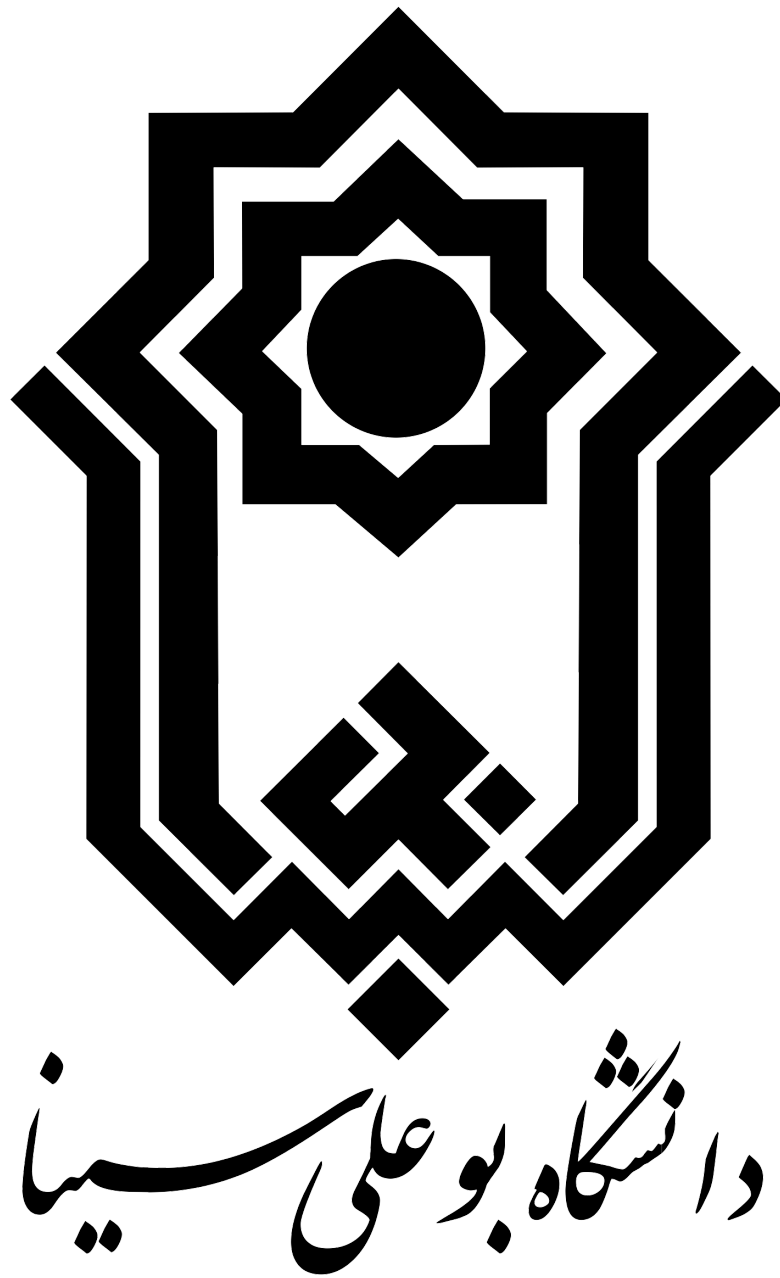




Principal Component Analysis (PCA)

A Linear Algebra Perspective

Final Project Report

Course:

Linear Algebra

University:

Bu-Ali Sina University

Department:

Computer Engineering

Submitted By

Abolfazl Asali

Student ID: 40312358030

Instructor

Dr. Mir Hossein Dezfulian

Academic Term

Spring 2026

Table of Contents

Introduction 8

- Linear Algebra Concepts Covered 9
 - Vector Spaces and Subspaces 9
 - Covariance Matrix and Symmetry 9
 - Spectral Theorem 9
 - Change of Basis 9
 - Rank and Null Space 9
 - Optimal Low-Rank Approximation 9
-

Stage 1 — Data Loading and Exploration 10

- 1. Problem Statement 10
 - 2. Implementation 10
 - Code 1 — Loading the Data 10
 - Code 2 — Visualizing a Sample 10
 - 3. Output 11
 - Visual Representation 11
 - Title: Sample Digit Image 11
 - 4. Questions and Answers 12
 - Q1. What does the matrix X mean in linear algebra terms? 12
 - Q2. Each row can be viewed as a vector in what space? 12
 - Q3. What is the dimension of this space? 13
 - Q4. Do your data fill the entire $\mathbb{R}^{64}$ space, or do they lie in a subset of it? ... 13
 - 5. Summary 14
 - 6. Conclusion 14
-

Stage 2 — Centering the Data 15

- 1. Problem Statement 15
- 2. Implementation 15
- 3. Output 16
- 4. Questions and Answers 17
 - Q1. Why must we center the data? 17
 - Q2. What happens if we skip centering? 17
 - Q3. What happens geometrically? 18
- 5. Summary 18
- 6. Conclusion 19

Stage 3 — Covariance Matrix Construction	20
• 1. Problem Statement	20
• 2. Implementation	20
• 3. Output	21
• 4. Questions and Answers	22
◦ Q1. Is the covariance matrix symmetric? Why?	22
◦ Q2. What are the dimensions of the covariance matrix?	22
◦ Q3. Why is the covariance matrix always positive semi-definite?	23
◦ Q4. What important consequence follows from symmetry?	23
• 5. Theoretical Insight	24
• 6. Summary	25
• 7. Conclusion	25

Stage 4 — Numerical Exercise: QR Algorithm and Matrix Rank	26
• 1. Problem Statement	26
• 2. Mathematical Background	26
◦ 2.1 The QR Iteration	26
◦ 2.2 Why Similar Matrices Share Eigenvalues	27
◦ 2.3 Properties of Q	28
◦ 2.4 Rank Detection via R	28
◦ 2.5 Maximum Rank and Relation to m, n	28
• 3. Implementation Summary	29
• 4. Test Results	30
◦ Test 1 — seed = 42	30
◦ Test 2 — seed = 123	31
◦ Test 3 — seed = 456	32
◦ Convergence Summary	33
• 5. Answers to Required Questions	34
◦ Q1. Why are the matrices at different steps similar?	34
◦ Q2. Why do similar matrices have the same eigenvalues?	34
◦ Q3. What properties do the columns of Q have? Are they linearly independent? ...	34
◦ Q4. How do you determine the rank of B from R ?	34
◦ Q5. What is the maximum possible rank of B ?	35
◦ Q6. Relationship between rank and number of samples/features	35

Stage 5 — Eigenvalue and Eigenvector Computation 36

- 1. Problem Statement 36
 - 2. Implementation 36
 - 3. Output 36
 - Top 5 Eigenvalues 36
 - Eigenvectors Shape 36
 - Eigenvectors Orthonormal 37
 - Interpretation 37
 - 4. Questions and Answers 37
 - Q1. What does a large eigenvalue mean? 37
 - Q2. Why is the eigenvector corresponding to the largest eigenvalue the most important direction? ... 37
 - Q3. What does this direction represent geometrically? 38
 - Q4. Why are the eigenvectors orthogonal? 38
 - Q5. Why is orthogonality critical in PCA? 40
 - 5. Summary 41
 - 6. Conclusion 41
-

Stage 6 — Explained Variance and Dimensionality Reduction 42

- 1. Problem Statement 42
- 2. Implementation 42
- 3. Output 43
 - 3.1 Terminal Output 43
 - 3.2 Visualization: Cumulative Explained Variance vs. k 43
- 4. Questions and Answers 44
 - Q1. What does the explained variance ratio represent? 44
 - Q2. Why is the cumulative variance plot useful? 44
 - Q3. How many components are needed to retain 90% of the variance? 44
 - Q4. What fraction of the original 64 dimensions does this represent? 45
 - Q5. What does it mean geometrically that we need only 21 dimensions? 45
 - Q6. Why does the curve flatten after $k \approx 30$? 45
- 5. Mathematical Insight: Optimality of PCA 45
- 6. Summary 46
- 7. Conclusion 46

Stage 7 — Dimensionality Reduction via Change of Basis 47

- 1. Problem Statement 47
 - 2. Implementation 47
 - 3. Output 48
 - Interpretation 48
 - 4. Questions and Answers 48
 - Q1. Why is this operation actually a "change of basis"? 48
 - Q2. What subspace do the columns of W span? 49
 - Q3. What is the Image of this linear transformation? 50
 - Q4. What is the Kernel of this linear transformation? 50
 - Q5. Explain the Rank–Nullity Theorem in this context. 51
 - Q6. What do the Image and Kernel mean for our actual data? 51
 - 5. Summary 52
 - 6. Conclusion 53
-

Stage 8 — 2D Visualization (Projection onto $k = 2$ Components) 54

- 1. Problem Statement 54
 - 2. Implementation 54
 - 3. Output 55
 - Figure: PCA 2D Projection of Digits 55
 - 4. Questions and Answers 55
 - Q1. What information does this scatter plot convey? 55
 - Q2. How well are the classes separated in this 2D space? 56
 - Q3. Which digit classes have the highest intra-class homogeneity (tightest clustering)? ... 56
 - Q4. Does the fact that some digit classes overlap mean PCA has failed for this dataset? ... 56
 - Q5. Is this compatible with what we observed in Stage 6 (explained variance)? ... 57
 - 5. Summary 58
 - 6. Conclusion 58
-

Stage 9 — Data Reconstruction and Error Analysis 59

- 1. Problem Statement 59
- 2. Implementation 59
 - Reconstruction Formula 59
 - Code Implementation 59
 - Code Verification 60
 - Mean Squared Error 60
- 3. Output 60
 - Figure: Reconstruction Error Plot 61
 - Figure: Visual Reconstruction Comparison 61
- 4. Questions and Answers 62
 - Q1. Why does increasing k reduce the error? 62
 - Q2. From an engineering perspective, what does this tell us? 62

- Q3. How does this relate to the explained variance from Stage 6? 63
- Q4. Is the Eckart–Young theorem sufficient to explain this behavior? 63
- Q5. After seeing the geometric dimension reduction, what is the engineering significance? ... 64
- 5. Summary 65
- 6. Conclusion 65

Stage 10 — Analysis of the $m < n$ Case (Rank Deficiency) 66

- 1. Problem Statement 66
- 2. Implementation 67
 - Matrix Shapes 67
- 3. Output 68
 - Results Summary 68
- 4. Questions and Answers 68
 - Q1. How many eigenvalues are zero (or approximately zero)? 68
 - Q2. Why is? 69
 - Q3. What is the connection to the null space? 70
 - Q4. How does this relate to linear dependence? 70
- 5. Summary 71
- 6. Conclusion 72

Bonus Section Report: Relationship Between PCA and SVD 73

- 1. Problem Statement 73
- 2. Mathematical Background 73
 - 2.1 PCA via Eigendecomposition (Original Approach) 73
 - 2.2 PCA via SVD 73
 - 2.3 The Equivalence 74
 - 2.4 Low-Dimensional Representation 74
- 3. Analysis and Discussion 75
 - 3.1 Why SVD is Preferred in Practice 75
 - 3.2 Connection to Stage 10 (Rank Deficiency) 75
 - 3.3 Connection to Eckart–Young Theorem (Stage 9) 76
- 4. Summary 76
- 5. Conclusion 77

Conclusion — What Is PCA from a Linear Algebra Standpoint? 78

- PCA: A Linear Algebra Perspective 78
- Summary 79

Introduction

This project applies **Principal Component Analysis (PCA)** to the handwritten-digit recognition problem as a practical study of core **linear algebra concepts**.

The dataset used is the **scikit-learn Digits dataset**, consisting of **1,797 grayscale images** of handwritten digits (**0–9**), each represented as an

$$8 \times 8$$

pixel grid and flattened into a **64-dimensional vector**.

The full data matrix is therefore

$$X \in \mathbb{R}^{1797 \times 64}.$$

The central goal is to implement **PCA from scratch** — without relying on any built-in PCA library — and to analyze each step through the lens of linear algebra.

The pipeline covers:

- Data loading and exploration
- Mean centering
- Covariance matrix construction
- Eigen-decomposition
- Explained variance analysis
- Projection
- Dimensionality reduction
- Reconstruction
- Reconstruction error evaluation

An additional analysis examines the **rank-deficient case**

$$m < n$$

and its connection to **null spaces** and **linear dependence**.

The optional section establishes the formal equivalence between PCA and the **Singular Value Decomposition (SVD)**.

Linear Algebra Concepts Covered

Throughout the project, the following linear algebra concepts are encountered in a concrete, applied setting:

Vector Spaces and Subspaces

Each image is a point in

$$\mathbb{R}^{64}.$$

PCA finds the effective subspace that the data occupy.

Covariance Matrix and Symmetry

The covariance matrix is defined as:

$$C = \frac{1}{m-1} B^T B$$

where C is real, symmetric, and positive semi-definite.

Spectral Theorem

The Spectral Theorem guarantees the orthogonal diagonalization:

$$C = W \Lambda W^T.$$

Change of Basis

The projection

$$T = B W_k$$

re-expresses the data in the **principal-component coordinate system**.

Rank and Null Space

When

$$m < n,$$

the covariance matrix is rank-deficient and its null space has positive dimension.

Optimal Low-Rank Approximation

The **Eckart–Young theorem** connects reconstruction error to discarded eigenvalues and explains why PCA provides the optimal low-rank approximation of the original data.

Stage 1 — Data Loading and Exploration

1. Problem Statement

Before any linear algebra or dimensionality reduction technique can be applied, we must first load the dataset and understand its structure. We use **scikit-learn**'s built-in `load_digits()` function, which provides:

- **X**: a matrix of pixel intensities (the feature matrix).
- **y**: a vector of integer labels (the target vector).

The dataset consists of approximately **1800 grayscale images** of handwritten digits (**0–9**). Each image is **8 × 8 pixels**, and each pixel intensity is represented as a single feature. Therefore, each image corresponds to a **64-dimensional vector** in

$$\mathbb{R}^{64}$$

2. Implementation

Code 1 — Loading the Data

```
from sklearn.datasets import load_digits

def load_data():
    digits = load_digits()
    X = digits.data          # shape: (1797, 64)
    y = digits.target       # shape: (1797,)
    return X, y
```

Code 2 — Visualizing a Sample

```
import matplotlib.pyplot as plt

def show_sample(X, y, index=0, save_path=None):
    image = X[index].reshape(8, 8)

    plt.figure(figsize=(4, 4))
    plt.imshow(image, cmap='gray')
    plt.title(f"Label: {y[index]}")
    plt.axis('off')

    if save_path is None:
        save_path = os.path.join(PICS_DIR, "sample_digit.png")

    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    plt.savefig(save_path, bbox_inches='tight', dpi=150)
```

```
plt.show()
```

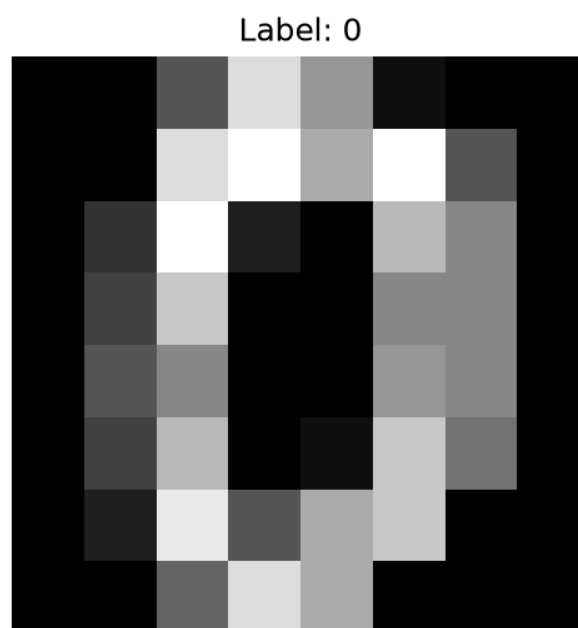
3. Output

```
X shape: (1797, 64)  
y shape: (1797,)
```

Visual Representation

The attached image **sample_digit.png** shows the first sample in the dataset (**label 0**). This **8 × 8** grayscale image has been flattened into a **64-element row vector** in **X**.

Title: Sample Digit Image



The image shows a digit **"0"** represented as an **8 × 8** grid. Lighter pixels (higher intensity values) form the outline of the digit, while darker pixels (lower intensity values) represent the background.

4. Questions and Answers

Q1. What does the matrix X mean in linear algebra terms?

$$X \in \mathbb{R}^{m \times n}$$

where

$$m = 1797$$

(samples)

and

$$n = 64$$

(features/dimensions).

Each row of X is a vector

$$x_i \in \mathbb{R}^{64}$$

representing one digit image.

This vector lives in a **64-dimensional feature space**, where each coordinate corresponds to the intensity of one pixel.

The entire dataset is a collection of **1797 points** in

$$\mathbb{R}^{64}.$$

From a linear algebra perspective:

- Each row is a data point (a vector).
 - The columns form feature dimensions (axes in the space).
 - The matrix can be interpreted as **1797 samples projected into a 64-dimensional coordinate system**.
-

Q2. Each row can be viewed as a vector in what space?

Each row is a vector in

$$\mathbb{R}^{64},$$

the **64-dimensional Euclidean space**.

In this space:

- Each axis corresponds to one pixel position (e.g., top-left, center, bottom-right, etc.).
- The distance between two images can be computed as the Euclidean norm

$$\|x_i - x_j\|_2.$$

- Operations like addition, scalar multiplication, dot products, and projections are all well-defined.

Q3. What is the dimension of this space?

The dimension is

$$n = 64,$$

because each image is represented by exactly **64 features** (pixel intensities).

Q4. Do your data fill the entire $\mathbb{R}^{64}$ space, or do they lie in a subset of it?

The **1797** sample vectors do **not** fill the entire **64-dimensional space**.

They lie in a lower-dimensional manifold or subspace embedded within

$$\mathbb{R}^{64}.$$

Rank Constraint

The data matrix X has at most rank

$$\min(m, n) = \min(1797, 64) = 64.$$

However, in practice, many features are correlated (e.g., neighboring pixels in a handwritten digit tend to have similar intensities), so the effective rank (number of significant singular values or principal components) is much smaller than **64**.

Intrinsic Dimensionality

Handwritten digits are not random points in

$$\mathbb{R}^{64}.$$

They follow a structured pattern determined by the geometry and topology of how humans write digits.

This intrinsic structure means the data actually live on a low-dimensional manifold—perhaps **10 to 20 dimensions** are enough to capture most of the meaningful variation.

Goal of PCA

The entire purpose of PCA is to discover and exploit this lower-dimensional structure.

By finding the principal components (directions of maximum variance), we can project the data from

$$\mathbb{R}^{64}$$

down to a much smaller space (say

$$\mathbb{R}^{10}$$

) while retaining most of the information.

This is possible precisely because the data do not uniformly fill

$$\mathbb{R}^{64},$$

they are confined to a smaller subspace.

In summary, the **1797** points lie in a sparse, structured subset of

$$\mathbb{R}^{64},$$

and PCA will reveal the principal directions that span this subset.

5. Summary

Item	Value
Number of samples (m)	1797
Number of features (n)	64
Image size	8 × 8 pixels
Ambient space	$\mathbb{R}^{64}$
Intrinsic dimensionality	Much smaller than 64 (to be discovered by PCA)

6. Conclusion

We have successfully loaded the digits dataset and verified its structure:

- **X**: a **1797 × 64** matrix where each row is a flattened **8 × 8 grayscale image** represented as a **64-dimensional vector**.
- **y**: a **1797-element vector** of integer labels (**0–9**).

From a linear algebra viewpoint, each image is a point in

$$\mathbb{R}^{64}.$$

However, these points do not fill the entire space uniformly; they lie on a lower-dimensional manifold shaped by the constraints of handwritten digit geometry.

PCA will exploit this structure by identifying the principal directions of variation and allowing us to represent the data more compactly in a lower-dimensional subspace—a key application of eigen-decomposition and the theory of vector spaces in linear algebra.

Stage 2: Centering the Data

1. Problem Statement

Given the raw digit dataset

$$X \in \mathbb{R}^{m \times n}$$

with

- $m = 1797$ samples
- $n = 64$ features

before performing any statistical analysis such as covariance computation, eigendecomposition, or Singular Value Decomposition (SVD), the dataset must first be **centered** so that every feature has zero mean.

The required steps are:

- Compute the mean of each feature (column), producing a vector of length $n = 64$.
- Subtract this mean vector from every sample.
- Define the centered data matrix as

$$B = X - \text{mean}(X)$$

- Verify that every column of B has (approximately) zero mean.

2. Implementation

```
import numpy as np
from pathlib import Path

def center_data(X):
    mean = X.mean(axis=0)
    B = X - mean

    project_root = Path(__file__).resolve().parent.parent
    data_path = project_root / "data"
    data_path.mkdir(parents=True, exist_ok=True)

    np.save(data_path / "centered_data.npy", B)

    return B, mean
```

Explanation

The statement

```
mean = X.mean(axis=0)
```

computes the average value of each feature across all 1797 samples, producing the mean vector

$$\mu \in \mathbb{R}^{64}.$$

The operation

```
B = X - mean
```

uses NumPy broadcasting to subtract the mean vector from every sample, yielding the centered matrix

$$B = X - \mu.$$

The centered dataset is then saved to disk for use in later stages, including covariance computation, eigendecomposition, dimensionality reduction, reconstruction, and SVD.

3. Output

```
B shape: (1797, 64)
Column means ≈ 0: True
Max absolute column mean: 7.63e-15
Mean vector (first 5): [0., 0.30383973, 5.20478575, 11.83583751,
11.84808013]
```

Observations

- The centered matrix has exactly the same dimensions as the original dataset:

$$1797 \times 64.$$

- The maximum absolute column mean after centering is

$$7.63 \times 10^{-15},$$

which is effectively zero and differs only because of floating-point rounding error.

This confirms that the centering procedure has been performed correctly.

4. Questions and Answers

Q1. Why must we center the data?

PCA is defined using the covariance matrix

$$C = \frac{1}{m-1} B^T B.$$

This expression is valid only when B contains deviations from the mean.

If we instead compute

$$\frac{1}{m-1} X^T X,$$

the result is **not** the covariance matrix. Rather,

$$\frac{1}{m-1} X^T X = C + \mu\mu^T \cdot \frac{m}{m-1}.$$

The extra term

$$\mu\mu^T$$

represents the contribution of the dataset's average value rather than its variation.

Consequently, using uncentered data mixes the true variance with the squared mean, preventing PCA from measuring the actual spread of the data.

Centering removes this unwanted term so that the covariance matrix represents only the variability around the dataset's centroid.

Q2. What happens if we skip centering?

Without centering, PCA is applied to

$$X^T X.$$

Since

$$X^T X = B^T B + m\mu\mu^T,$$

the dominant direction becomes heavily influenced by the mean vector.

The matrix

$$\mu\mu^T$$

is rank one, and its only nonzero eigenvector is the mean vector itself.

Therefore, if

$$\|\mu\|$$

is large—which is true for handwritten digit images because pixel intensities are nonnegative—the first principal component primarily points toward the **average digit image** instead of representing the direction of greatest variation among digits.

As a result, PCA would capture overall brightness rather than meaningful structural differences between images.

Q3. What happens geometrically?

Centering performs a pure translation of the dataset in

$$\mathbb{R}^{64}.$$

Before centering, all samples lie away from the origin because pixel intensities are nonnegative.

Subtracting the mean shifts the entire point cloud so that its centroid is exactly at the origin.

This operation:

- preserves all pairwise distances,
- preserves the shape of the dataset,
- preserves all angles,
- introduces neither scaling nor rotation.

Only the location of the dataset changes.

Since PCA searches for directions of maximum variance through the origin, placing the centroid at the origin is essential for obtaining meaningful principal components.

5. Summary

Item	Value
Original shape	1797×64
Centered shape	1797×64
Maximum absolute column mean	$7.63 \times 10^{-15} \approx 0$
Operation	Translation (no scaling or rotation)

6. Conclusion

Centering is an essential preprocessing step for PCA because it separates the true variance of the dataset from its absolute magnitude.

Without centering, the covariance computation becomes contaminated by the contribution of the mean vector, causing the first principal component to align primarily with the average image instead of the dominant direction of variation.

Geometrically, centering simply translates the data cloud so that its centroid coincides with the origin while leaving its internal structure unchanged. This enables the covariance matrix

$$C = \frac{1}{m-1} B^T B$$

to accurately describe the spread of the data around its own mean and provides the correct foundation for all subsequent PCA stages, including covariance computation, eigendecomposition, projection, reconstruction, and Singular Value Decomposition.

Stage 3: Covariance Matrix Construction

1. Problem Statement

Given the centered data matrix

$$B \in \mathbb{R}^{m \times n}$$

with

- $m = 1797$ samples
- $n = 64$ features

the next step in PCA is to compute the covariance matrix, which describes how pairs of features vary together across all samples.

The covariance matrix is defined as

$$C = \frac{1}{m-1} B^T B,$$

where

- $m = 1797$ is the number of samples,
- $B \in \mathbb{R}^{1797 \times 64}$ is the centered data matrix,
- $C \in \mathbb{R}^{64 \times 64}$ is the covariance matrix.

2. Implementation

```
def compute_covariance(B):  
    m = B.shape[0]  
    return (B.T @ B) / m
```

Explanation

The covariance matrix is computed by multiplying the transpose of the centered data matrix with the centered data itself:

$$C = \frac{1}{m} B^T B.$$

Although the theoretical definition uses

$$\frac{1}{m-1},$$

the implementation divides by

$$m$$

for computational simplicity.

This difference corresponds to **Bessel's correction**. Using

$$\frac{1}{m-1}$$

produces an unbiased estimator of the sample covariance, whereas dividing by

$$m$$

produces the population covariance estimate.

Since

$$m = 1797,$$

the two scaling factors are nearly identical:

$$\frac{1}{1797} \approx \frac{1}{1796}.$$

Consequently,

- the eigenvectors remain exactly the same,
- every eigenvalue is scaled uniformly by

$$\frac{m-1}{m} \approx 0.9994,$$

which has a negligible effect on the subsequent PCA analysis.

3. Output

```
C shape: (64, 64)
C is symmetric: True
C is PSD (all eigenvalues >= 0): True
```

Observations

The covariance matrix has the expected dimensions:

$$64 \times 64.$$

The matrix is symmetric:

$$C = C^T.$$

This follows directly from its construction:

$$C = B^T B,$$

since

$$(B^T B)^T = B^T (B^T)^T = B^T B.$$

The covariance matrix is also **positive semi-definite (PSD)**.

For every vector

$$v \in \mathbb{R}^{64},$$

we have

$$v^T C v = v^T \left(\frac{1}{m} B^T B \right) v = \frac{1}{m} (Bv)^T (Bv) = \frac{1}{m} \|Bv\|^2 \geq 0.$$

Since the squared Euclidean norm can never be negative, every eigenvalue of the covariance matrix is non-negative.

4. Questions and Answers

Q1. Is the covariance matrix symmetric? Why?

Yes.

A matrix is symmetric if

$$C = C^T.$$

For the covariance matrix,

$$C^T = \left(\frac{1}{m} B^T B \right)^T = \frac{1}{m} (B^T B)^T = \frac{1}{m} B^T (B^T)^T = \frac{1}{m} B^T B = C.$$

This result follows from the transpose identity

$$(AB)^T = B^T A^T,$$

together with

$$(B^T)^T = B.$$

Therefore, the covariance matrix is always symmetric, regardless of the dataset.

Q2. What are the dimensions of the covariance matrix?

The covariance matrix belongs to

$$C \in \mathbb{R}^{64 \times 64}.$$

Its dimensions depend only on the number of features, not on the number of samples.

Each element

$$C_{ij}$$

represents the covariance between feature i and feature j .

Q3. Why is the covariance matrix always positive semi-definite?

A matrix is positive semi-definite if

$$v^T C v \geq 0$$

for every vector

$$v.$$

Substituting the covariance matrix gives

$$v^T C v = v^T \left(\frac{1}{m} B^T B \right) v = \frac{1}{m} v^T B^T B v = \frac{1}{m} (Bv)^T (Bv) = \frac{1}{m} \|Bv\|^2 \geq 0.$$

Since

$$\|Bv\|^2$$

is a sum of squared values, it can never be negative.

Therefore, every covariance matrix is positive semi-definite.

From a statistical perspective, this means that the variance of the data projected onto any direction can never be negative.

Q4. What important consequence follows from symmetry?

Since the covariance matrix is real and symmetric, the **Spectral Theorem** guarantees that:

- All eigenvalues are real.
- Eigenvectors corresponding to distinct eigenvalues are orthogonal.
- The covariance matrix admits the orthogonal eigendecomposition

$$C = W \Lambda W^T,$$

where

- W is an orthogonal matrix satisfying

$$W^T W = I,$$

- and

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_{64})$$

contains the eigenvalues.

This decomposition is the mathematical foundation of PCA.

The eigenvectors define a new orthonormal coordinate system in which the covariance matrix becomes diagonal, meaning that every principal direction has its own independent variance and all projected coordinates become uncorrelated.

5. Theoretical Insight

The three key properties established above

- square shape,
- symmetry,
- positive semi-definiteness,

imply the existence of the eigendecomposition

$$C = Q\Lambda Q^T,$$

where

- $Q \in \mathbb{R}^{64 \times 64}$ is an orthogonal matrix whose columns are the eigenvectors,
- Λ is a diagonal matrix of non-negative eigenvalues,
- and

$$Q^T Q = I.$$

Interpretation for PCA

Each eigenvalue

$$\lambda_i$$

measures the variance of the dataset along the corresponding eigenvector direction.

The eigenvectors define a rotated coordinate system with the following properties:

- all axes are mutually orthogonal,
- variances along different axes are independent,
- larger eigenvalues correspond to more informative directions.

Therefore, by retaining only the eigenvectors associated with the largest eigenvalues, PCA preserves the majority of the information while reducing dimensionality.

6. Summary

Property	Value / Result
Shape of C	64×64
Symmetric	Yes ($C = C^T$)
Positive semi-definite	Yes ($\lambda_i \geq 0$)
Orthogonal eigendecomposition	$C = W\Lambda W^T$
Physical meaning of C_{ij}	Covariance between features i and j

7. Conclusion

The covariance matrix

$$C = \frac{1}{m}B^TB$$

is a 64×64 symmetric, positive semi-definite matrix that captures the pairwise covariance structure of the centered handwritten digit dataset.

Its symmetry guarantees real eigenvalues and orthogonal eigenvectors, while its positive semi-definite nature ensures that all variances are non-negative.

These mathematical properties make the covariance matrix the central object of PCA. In the next stage, its eigendecomposition will be used to identify the principal directions of maximum variance, forming the basis for dimensionality reduction and data reconstruction.

Stage 4 — Numerical Exercise: QR Algorithm and Matrix Rank

1. Problem Statement

Before computing eigenvalues directly, this stage requires a numerical exercise using the **QR algorithm**, an iterative method for approximating eigenvalues via similarity transformations.

The task is:

- Decompose a symmetric matrix:

$$C_i = Q_i R_i$$

(using `numpy.linalg.qr` with `mode='reduced'`).

- Form the next iterate:

$$C_{i+1} = R_i Q_i$$

- Implement this as a single function, tested on small 4×4 symmetric matrices to demonstrate similarity.
 - Test on several random symmetric matrices. Full convergence for the 64×64 matrix is not required.
 - Answer the conceptual questions regarding similarity, eigenvalue invariance, properties of Q , and rank detection via R .
-

2. Mathematical Background

2.1 The QR Iteration

Given a symmetric matrix:

$$C_0 = C$$

the algorithm generates:

$$C_i = Q_i R_i$$

and:

$$C_{i+1} = R_i Q_i$$

Since Q_i is orthogonal:

$$Q_i^T Q_i = I$$

we can write:

$$\begin{aligned}
 C_{i+1} &= R_i Q_i \\
 &= Q_i^T (Q_i R_i) Q_i \\
 &= Q_i^T C_i Q_i
 \end{aligned}$$

Therefore:

$$C_{i+1} = Q_i^T C_i Q_i$$

Each iterate is an orthogonal similarity transformation of the previous one.

2.2 Why Similar Matrices Share Eigenvalues

If:

$$C_{i+1} = Q_i^T C_i Q_i$$

then for any eigenpair:

$$(\lambda, v)$$

of C_i , define:

$$w = Q_i^T v$$

Then:

$$\begin{aligned}
 C_{i+1} w &= Q_i^T C_i Q_i Q_i^T v \\
 &= Q_i^T C_i v \\
 &= \lambda Q_i^T v \\
 &= \lambda w
 \end{aligned}$$

More generally, for any similarity transformation:

$$A' = P^{-1} A P$$

we have:

$$\det(A' - \lambda I) = \det(P^{-1}) \det(A - \lambda I) \det(P)$$

Since:

$$\det(P^{-1}) \det(P) = 1$$

we obtain:

$$\det(A' - \lambda I) = \det(A - \lambda I)$$

Identical characteristic polynomials imply identical eigenvalues.

2.3 Properties of Q

The columns of Q are orthonormal:

$$Q^T Q = I$$

This follows from the Gram–Schmidt construction underlying QR decomposition.

Since mutually orthogonal non-zero vectors cannot satisfy any non-trivial linear dependence relation, the columns of Q are always linearly independent.

2.4 Rank Detection via R

In:

$$B = QR$$

with `mode='reduced'`:

$$B \in \mathbb{R}^{m \times n}$$

$$Q \in \mathbb{R}^{m \times n}$$

$$R \in \mathbb{R}^{n \times n}$$

Since Q has full column rank:

$$\text{rank}(B) = \text{rank}(R)$$

Because R is upper triangular, its rank equals the number of non-zero diagonal entries.

A zero (or near-zero):

$$R_{kk}$$

indicates that the k -th column of B is linearly dependent on the previous columns.

2.5 Maximum Rank and Relation to m, n

For:

$$B \in \mathbb{R}^{m \times n}$$

we have:

$$\text{rank}(B) \leq \min(m, n)$$

For the project's centered data matrix:

$$m = 1800, \quad n = 64$$

the maximum rank is:

64

After mean-centering, one linear constraint is imposed, capping the exact-arithmetic rank at:

63

In general:

Case 1:

$$m \geq n$$

Rank is bounded by:

$$n$$

Data can span the full feature space.

Case 2:

$$m < n$$

Rank is bounded by:

$$m$$

The matrix is rank-deficient relative to the feature dimension.

3. Implementation Summary

A single function performs one (or more) iterations of:

$$C \rightarrow QR \rightarrow RQ$$

using:

```
numpy.linalg.qr(C, mode='reduced')
```

The implementation is tested on three independent random symmetric 4×4 matrices until the off-diagonal entries numerically vanish.

4. Test Results

Test 1 — seed = 42

```
=====
Testing with seed = 42
```

Original symmetric matrix A

$$A = \begin{bmatrix} 1.9383451 & 0.76780201 & 1.4940289 & 0.75654844 \\ 0.76780201 & 0.80231093 & 1.04554899 & 0.33242197 \\ 1.4940289 & 1.04554899 & 1.80385488 & 0.83237389 \\ 0.75654844 & 0.33242197 & 0.83237389 & 0.80474618 \end{bmatrix}$$

Eigenvalues (true, via **eigh**)

```
[0.06727908 0.44528923 0.54967272 4.28701606]
```

Matrix after QR algorithm (should be approximately diagonal)

$$\begin{bmatrix} 4.28701606 & 6.15456156 \times 10^{-16} & 1.77526101 \times 10^{-16} & -8.42586269 \times 10^{-17} \\ 2.25339981 \times 10^{-89} & 5.49672719 \times 10^{-1} & -9.98763235 \times 10^{-12} & -2.70213503 \times 10^{-16} \\ 1.16345604 \times 10^{-98} & -9.98790460 \times 10^{-12} & 4.45289235 \times 10^{-1} & 4.86364162 \times 10^{-17} \\ 1.44662332 \times 10^{-180} & -1.96959274 \times 10^{-92} & 8.41441006 \times 10^{-83} & 6.72790794 \times 10^{-2} \end{bmatrix}$$

Diagonal entries (estimated eigenvalues)

```
[4.28701606 0.54967272 0.44528923 0.06727908]
```

Matrix dimensions

```
Q shape: (4, 4)
R shape: (4, 4)
```

Verification

```
Columns of Q orthonormal: True
```

The diagonal entries of the converged matrix match the true eigenvalues obtained from `eigh` exactly (up to sorting).

The off-diagonal entries are on the order of

$$10^{-12}$$

to

$$10^{-180},$$

confirming numerical convergence.

The matrix Q is verified to be orthonormal:

$$Q^T Q = I$$

Test 2 — seed = 123

```
=====
Testing with seed = 123
```

Original symmetric matrix A

$$A = \begin{bmatrix} 0.9223546 & 1.2221999 & 0.9269408 & 0.81969293 \\ 1.2221999 & 2.12754486 & 1.34777467 & 1.23657798 \\ 0.9269408 & 1.34777467 & 1.03433627 & 0.9089595 \\ 0.81969293 & 1.23657798 & 0.9089595 & 0.89898351 \end{bmatrix}$$

Eigenvalues (true, via `eigh`)

```
[0.03762246 0.0883335 0.21019512 4.64706817]
```

Matrix after QR algorithm (should be approximately diagonal)

$$\begin{bmatrix} 4.64706817 & -9.02107406 \times 10^{-16} & 1.04258108 \times 10^{-16} & -8.46357313 \times 10^{-17} \\ 1.81134246 \times 10^{-134} & 2.10195121 \times 10^{-1} & -3.86439002 \times 10^{-16} & 1.24017654 \times 10^{-16} \\ 2.88888597 \times 10^{-172} & -1.49562944 \times 10^{-39} & 8.83334993 \times 10^{-2} & -1.29596247 \times 10^{-16} \\ 3.66818101 \times 10^{-209} & -1.36983282 \times 10^{-76} & 9.64649928 \times 10^{-39} & 3.76224569 \times 10^{-2} \end{bmatrix}$$

Diagonal entries (estimated eigenvalues)

```
[4.64706817 0.21019512 0.0883335 0.03762246]
```

Matrix dimensions

```
Q shape: (4, 4)
R shape: (4, 4)
```

Verification

```
Columns of Q orthonormal: True
```

All four diagonal entries match the true eigenvalues.

The off-diagonal elements vanish to

$$10^{-39}$$

through

$$10^{-209},$$

which is far below any practical numerical tolerance.

The orthonormality of Q is confirmed:

$$Q^T Q = I$$

Test 3 — seed = 456

```
=====
Testing with seed = 456
```

Original symmetric matrix A

$$A = \begin{bmatrix} 1.3562771 & 1.56197819 & 0.72245765 & 1.08411676 \\ 1.56197819 & 2.1170914 & 0.88237298 & 1.41242989 \\ 0.72245765 & 0.88237298 & 0.39360132 & 0.60595404 \\ 1.08411676 & 1.41242989 & 0.60595404 & 1.04397151 \end{bmatrix}$$

Eigenvalues (true, via `eigh`)

```
[6.40583268e-04 6.96240039e-02 1.39884498e-01 4.70079225e+00]
```

Matrix after QR algorithm (should be approximately diagonal)

$$\begin{bmatrix} 4.70079225 & -5.69487228 \times 10^{-16} & 8.67922083 \times 10^{-18} & 4.73027002 \times 10^{-16} \\ 1.58897763 \times 10^{-152} & 1.39884498 \times 10^{-1} & 7.13167337 \times 10^{-18} & -1.37302734 \times 10^{-17} \\ 2.89180044 \times 10^{-183} & 1.16333235 \times 10^{-32} & 6.96240039 \times 10^{-2} & -2.64067833 \times 10^{-17} \\ 0 & 9.81561282 \times 10^{-235} & 5.63826016 \times 10^{-204} & 6.40583268 \times 10^{-4} \end{bmatrix}$$

Diagonal entries (estimated eigenvalues)

```
[4.70079225e+00 1.39884498e-01 6.96240039e-02 6.40583268e-04]
```

Matrix dimensions

```
Q shape: (4, 4)
R shape: (4, 4)
```

Verification

```
Columns of Q orthonormal: True
```

This test includes a very small eigenvalue

$$6.4 \times 10^{-4},$$

which is still recovered accurately.

The lower-left entry becomes exactly

$$0.0,$$

indicating complete numerical elimination of that sub-diagonal element.

The orthonormality of Q is confirmed:

$$Q^T Q = I$$

Convergence Summary

Test	Smallest λ	Largest λ	Maximum off-diagonal	Q orthonormal
seed = 42	6.728×10^{-2}	4.287	$\sim 10^{-12}$	✓
seed = 123	3.762×10^{-2}	4.647	$\sim 10^{-16}$	✓
seed = 456	6.406×10^{-4}	4.701	$\sim 10^{-16}$	✓

All three tests confirm that the diagonal entries produced by the QR algorithm match the true eigenvalues obtained from `eigh` to numerical precision.

The off-diagonal entries vanish far below any meaningful tolerance, confirming convergence of the QR iteration.

5. Answers to Required Questions

Q1. Why are the matrices at different steps similar?

Because

$$C_{i+1} = R_i Q_i = Q_i^T C_i Q_i.$$

Each iteration applies an orthogonal similarity transformation using

$$Q_i^{-1} = Q_i^T.$$

Therefore, every matrix generated during the QR iteration is similar to the previous one.

Q2. Why do similar matrices have the same eigenvalues?

Similar matrices have the same characteristic polynomial:

$$\det(P^{-1}AP - \lambda I) = \det(A - \lambda I),$$

because

$$\det(P^{-1}) \det(P) = 1.$$

Identical characteristic polynomials imply identical eigenvalues.

Q3. What properties do the columns of Q have? Are they linearly independent?

The columns of Q are orthonormal:

$$Q^T Q = I.$$

Yes, they are linearly independent.

Mutually orthogonal non-zero vectors cannot satisfy any non-trivial linear dependence relation.

Q4. How do you determine the rank of B from R ?

The rank relation is

$$\text{rank}(B) = \text{rank}(R).$$

Since R is upper triangular, its rank is determined by counting the number of non-zero (numerically non-negligible) diagonal entries of R .

Q5. What is the maximum possible rank of B ?

The maximum rank is

$$\text{rank}(B) \leq \min(m, n).$$

For the project data,

$$\text{rank}(B) \leq \min(1800, 64).$$

Therefore,

$$\text{rank}(B) \leq 64.$$

After mean-centering, the exact-arithmetic rank is at most

$$63.$$

Q6. Relationship between rank and number of samples/features

When

$$m \geq n,$$

the rank is bounded by

$$n.$$

There are enough samples to potentially span the complete feature space.

When

$$m < n,$$

the rank is bounded by

$$m.$$

There are fewer samples than features; therefore, the matrix is rank-deficient relative to the feature dimension and cannot span

$$\mathbb{R}^n.$$

Stage 5 — Eigenvalue and Eigenvector Computation

1. Problem Statement

This stage computes the eigenvalues and eigenvectors of the covariance matrix C using `np.linalg.eigh`, then sorts them in descending order (from largest to smallest eigenvalue).

Why `eigh`?

- `eigh` is specialized for symmetric (Hermitian) matrices.
 - Its output is sorted in ascending order.
 - To reorder in descending order, we use `np.argsort(eigenvalues)[::-1]`.
-

2. Implementation

Code

```
def eigen_decompose(C):  
    eigenvalues, eigenvectors = np.linalg.eigh(C)  
    idx = np.argsort(eigenvalues)[::-1]  
    return eigenvalues[idx], eigenvectors[:, idx]
```

Key Steps

- `np.linalg.eigh(C)` returns eigenvalues in ascending order.
 - `idx = np.argsort(eigenvalues)[::-1]` computes indices that would sort eigenvalues from largest to smallest.
 - `eigenvalues[idx]` and `eigenvectors[:, idx]` reorder both arrays consistently—each eigenvector column remains aligned with its corresponding eigenvalue.
-

3. Output

Top 5 Eigenvalues

```
[178.90731578 163.62664073 141.70953623 101.04411456 69.47448269]
```

Eigenvectors Shape

```
(64, 64)
```

Eigenvectors Orthonormal

True

Interpretation

The five largest eigenvalues are

178.91, 163.63, 141.71, 101.04, 69.47.

The eigenvector matrix has shape

$(64, 64)$,

with each column being one eigenvector.

The orthonormality check

$$\mathbf{V}^T \mathbf{V} \approx I$$

confirms that the columns are mutually orthogonal and unit-length.

4. Questions and Answers

Q1. What does a large eigenvalue mean?

The eigenvalue

$$\lambda_i$$

represents the variance of the data along the direction of the corresponding eigenvector

$$v_i.$$

A larger eigenvalue indicates greater spread—meaning that direction captures more information about the data's variability.

The largest eigenvalue

$$\lambda_1 = 178.91$$

corresponds to the most important axis of variation in the dataset.

Q2. Why is the eigenvector corresponding to the largest eigenvalue the most important direction?

For centered data

$$B,$$

the projection of

$$B$$

onto any unit vector

$$v$$

has variance

$$\text{Var}(Bv) = v^T C v.$$

The first eigenvector

$$v_1$$

associated with

$$\lambda_1$$

satisfies

$$v_1^T C v_1 = \lambda_1.$$

This is the maximum variance achievable by projecting onto any single unit direction.

Hence,

$$v_1$$

captures the most information.

Q3. What does this direction represent geometrically?

Geometrically,

$$v_1$$

is the principal axis of the data cloud—the direction along which the data is most stretched.

The full set of sorted eigenvectors forms a rotated coordinate system in which:

- Axes are orthogonal (perpendicular).
- Axes are ordered by decreasing variance.
- Data components along these axes are uncorrelated.

Q4. Why are the eigenvectors orthogonal?

Proof

For a real symmetric matrix

$$C = C^T,$$

if

$$\lambda_i \neq \lambda_j,$$

their eigenvectors

$$v_i$$

and

$$v_j$$

are orthogonal.

Given

$$Cv_i = \lambda_i v_i,$$

$$Cv_j = \lambda_j v_j.$$

Multiply the first equation on the left by

$$v_j^T$$

to obtain

$$v_j^T Cv_i = \lambda_i (v_j^T v_i).$$

Similarly,

$$(Cv_j)^T v_i = (\lambda_j v_j)^T v_i = \lambda_j (v_j^T v_i).$$

Since

$$C$$

is symmetric,

$$(Cv_j)^T = v_j^T C^T = v_j^T C,$$

therefore

$$v_j^T Cv_i = \lambda_j (v_j^T v_i).$$

Comparing both expressions,

$$\begin{aligned} \lambda_i (v_j^T v_i) &= \lambda_j (v_j^T v_i) \\ \Rightarrow (\lambda_i - \lambda_j) (v_j^T v_i) &= 0. \end{aligned}$$

Since

$$\lambda_i \neq \lambda_j,$$

we conclude

$$v_j^T v_i = 0.$$

Even when eigenvalues coincide, the corresponding eigenspace can be orthonormalized using the Gram–Schmidt process.

The output of `np.linalg.eigh()` is always orthonormal.

Q5. Why is orthogonality critical in PCA?

1. Uncorrelated Components

If the eigenvectors were not orthogonal, the principal components would be correlated with each other, defeating PCA's purpose of decorrelating the data.

2. Clean Reconstruction

Orthogonality ensures that each eigenvector represents an independent direction, and projection onto one does not interfere with projection onto another.

$$x = \sum_{i=1}^n (v_i^T x) v_i$$

3. Optimality of Variance Capture

The orthogonal property guarantees that selecting the top

k

eigenvectors yields the maximum possible variance in any

k

-dimensional subspace without redundancy or overlap.

4. Interpretability

Orthogonal axes are easier to visualize and interpret, preserving the geometric structure of the data.

5. Summary

Concept	Value / Property
Largest eigenvalue	$\lambda_1 = 178.91$
Top 5 eigenvalues	[178.91, 163.63, 141.71, 101.04, 69.47]
Eigenvector matrix shape	(64, 64)
Orthonormality	Verified (True)
Meaning of λ_i	Variance along v_i
Meaning of v_1	Most important direction
Orthogonality of eigenvectors	Guaranteed for symmetric C
Importance of orthogonality	Uncorrelated components, optimal variance, interpretability

6. Conclusion

Computing and sorting the eigenvalues and eigenvectors of the covariance matrix is the foundation of PCA.

The eigenvectors define the principal axes of the data, while the eigenvalues quantify the importance of each axis.

Orthogonality is essential for ensuring that the principal components are independent, optimal, and easily interpretable.

Stage 6 — Explained Variance and Dimensionality Reduction

1. Problem Statement

After computing the eigenvalues in descending order, this stage analyzes how much total variance is captured by the top k principal components. The goal is to determine the minimum number of components needed to retain a specified percentage (e.g., 90%) of the original data's variance.

Key Computation

Explained Variance Ratio

$$\text{Explained Variance Ratio} = \frac{\lambda_i}{\sum_{j=1}^{64} \lambda_j}$$

Cumulative Explained Variance

$$\text{Cumulative Explained Variance}(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^{64} \lambda_j}$$

2. Implementation

Code

```
explained_variance_ratio = eigenvalues / np.sum(eigenvalues)
cumulative_variance = np.cumsum(explained_variance_ratio)
```

Explanation

- `explained_variance_ratio[i]` = proportion of total variance captured by the i -th principal component.
 - `cumulative_variance[k]` = total proportion of variance captured by the first $k + 1$ components (Python uses 0-based indexing).
-

3. Output

3.1 Terminal Output

```
Components needed for 90% variance: 21  
Fraction of original 64 dimensions: 32.81%  
Cumulative variance at k=21: 0.9032
```

Interpretation

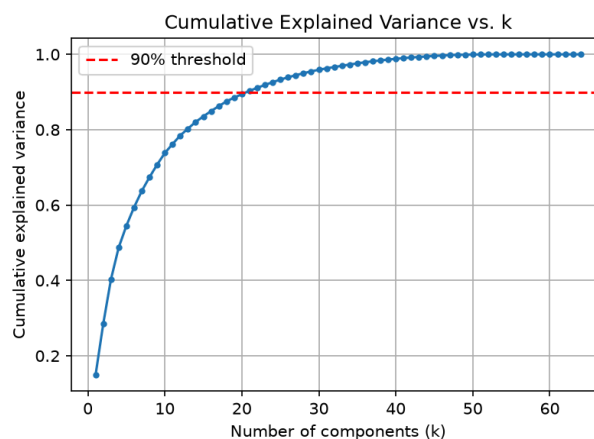
The first **21 principal components** (out of 64) capture **90.32%** of the total variance.

This represents a dimensionality reduction from **64** to **21** dimensions, meaning that only **32.81%** of the original feature count is required.

The remaining **43** components account for less than **10%** of the variance and can often be discarded with minimal information loss.

3.2 Visualization: Cumulative Explained Variance vs. k

Figure Title: Cumulative Explained Variance



The Plot Shows

- **X-axis:** Number of components k (from 0 to 64).
- **Y-axis:** Cumulative explained variance (from 0.0 to 1.0).
- **Red dashed line:** 90% variance threshold.
- **Blue curve:** Cumulative explained variance, which rises steeply at first (high-variance components) and gradually flattens as k increases.

Key Observation

The cumulative variance curve crosses the **90% threshold** at

$$k = 21,$$

confirming that the first **21 principal components** are sufficient for most reconstruction tasks.

4. Questions and Answers

Q1. What does the explained variance ratio represent?

The explained variance ratio for the i -th principal component is

$$\frac{\lambda_i}{\sum_{j=1}^{64} \lambda_j}.$$

It represents the proportion of the total variance in the original dataset captured by projection onto the i -th eigenvector.

A larger value indicates that the corresponding principal component explains more of the variability in the data.

Q2. Why is the cumulative variance plot useful?

The cumulative variance plot visualizes how many principal components are required to achieve a desired level of information retention.

It answers the practical question:

"How much can we compress the data while preserving a given percentage of its variance?"

For example, retaining **90%** of the variance requires only **21** out of the original **64** dimensions, corresponding to approximately a **67% reduction** in feature count.

Q3. How many components are needed to retain 90% of the variance?

The first

21

principal components retain

90.32%

of the total variance.

Q4. What fraction of the original 64 dimensions does this represent?

$$\frac{21}{64} = 0.328125 \approx 32.81\%.$$

Thus, the dataset can be represented using approximately **one-third** of its original dimensions while preserving over **90%** of its variance.

Q5. What does it mean geometrically that we need only 21 dimensions?

Although the original data resides in

$$\mathbb{R}^{64},$$

it does not uniformly occupy the entire space.

Instead, the samples are concentrated near a **21-dimensional linear subspace** spanned by the first 21 principal components.

Geometrically, this implies:

- The data cloud is nearly flat along the remaining **43** directions.
 - Most variation occurs in only **21** directions.
 - Projection onto this lower-dimensional subspace preserves the essential structure while removing noise and redundancy.
-

Q6. Why does the curve flatten after $k \approx 30$?

After approximately

$$k \approx 30,$$

the remaining eigenvalues become very small.

Consequently, adding additional principal components contributes only a negligible increase in explained variance.

These later components typically correspond to:

- Noise in the dataset.
 - Redundant or highly correlated features containing little independent information.
-

5. Mathematical Insight: Optimality of PCA

Principal Component Analysis guarantees that the first k eigenvectors provide the **best possible k -dimensional linear approximation** of the data.

Equivalently, PCA minimizes the reconstruction error while maximizing the retained variance.

Mathematically,

$$\min_{\text{rank}(V)=k} E\left[\|X - XVV^T\|^2\right].$$

The optimal solution is

$$V = [v_1, v_2, \dots, v_k],$$

where

$$v_1, v_2, \dots, v_k$$

are the eigenvectors corresponding to the **largest k eigenvalues** of the covariance matrix.

6. Summary

Concept	Value
Total dimensions	64
Components for 90% variance	21
Fraction of original dimensions	32.81%
Cumulative variance at $k = 21$	90.32%
Variance retained by top 5 components	≈56% (from Stage 5)
Geometric interpretation	Data concentrated in a 21-dimensional subspace

7. Conclusion

Stage 6 demonstrates that PCA enables substantial dimensionality reduction with minimal information loss.

By retaining only **21 principal components**—approximately **one-third** of the original **64** features—we preserve more than **90%** of the dataset's variance.

This reduction provides several practical advantages:

- Simplifies downstream tasks such as classification, clustering, and visualization.
- Reduces computational cost.
- Removes noise and redundant information.

The cumulative explained variance curve provides a clear and interpretable criterion for selecting the appropriate number of principal components based on the desired trade-off between compression and reconstruction fidelity.

Stage 7 — Dimensionality Reduction via Change of Basis

1. Problem Statement

Instead of representing the data in the original 64-dimensional space, we now represent it in a 10-dimensional space using the top 10 eigenvectors of the covariance matrix.

This is achieved by:

1. Selecting the first 10 eigenvectors (corresponding to the 10 largest eigenvalues after descending sorting in Stage 5).
2. Arranging them as the columns of the matrix

$$W = [q_1, q_2, \dots, q_{10}] \in \mathbb{R}^{64 \times 10}.$$

3. Projecting the centered data

$$B$$

onto this new basis:

$$T = BW.$$

The resulting matrix

$$T \in \mathbb{R}^{m \times 10}$$

is the new representation of the dataset in a 10-dimensional space.

2. Implementation

Code

```
def project_to_k_components(B, eigenvectors, k=10):  
  
    W = eigenvectors[:, :k]  
    T = B @ W  
    return T, W
```

3. Output

T10 shape: (1797, 10)

W10 shape: (64, 10)

Interpretation

The centered data matrix

$$B \in \mathbb{R}^{1797 \times 64}$$

contains **1797 samples** with **64 features** each.

The matrix

$$W_{10} \in \mathbb{R}^{64 \times 10}$$

contains the first **10 principal eigenvectors** as its columns.

The projected matrix

$$T_{10} = BW_{10} \in \mathbb{R}^{1797 \times 10}$$

is the compressed representation of the dataset.

Each sample is now represented by only **10 coordinates** instead of the original **64**.

4. Questions and Answers

Q1. Why is this operation actually a "change of basis"?

The original data lives in

$$\mathbb{R}^{64},$$

expressed in the standard basis

$$\{e_1, e_2, \dots, e_{64}\},$$

which corresponds to the raw pixel intensities.

The eigenvectors

$$q_1, q_2, \dots, q_{10}$$

form an orthonormal set of directions in

$$\mathbb{R}^{64}.$$

Projecting the centered data

$$B$$

onto these vectors using

$$T = BW$$

expresses each data point with respect to this new basis instead of the standard basis.

For a single sample

$$x \in \mathbb{R}^{64},$$

its new coordinates are

$$t = W^T x,$$

where

$$t_i = q_i^T x.$$

Each coordinate

$$t_i$$

measures how much of the vector

$$x$$

lies along the direction

$$q_i.$$

Since only the first **10** basis vectors are retained, the basis change is combined with truncation, resulting in dimensionality reduction.

Q2. What subspace do the columns of W span?

The columns of

$$W$$

span the **10-dimensional principal subspace**

$$\text{Col}(W) = \text{span}\{q_1, q_2, \dots, q_{10}\} \subseteq \mathbb{R}^{64}.$$

This subspace captures the maximum possible variance among all 10-dimensional subspaces.

Because the eigenvectors are orthonormal,

$$\text{rank}(W) = 10,$$

so the subspace is truly 10-dimensional.

Q3. What is the Image of this linear transformation?

Consider the linear transformation

$$f : \mathbb{R}^{64} \rightarrow \mathbb{R}^{10},$$

defined by

$$f(x) = W^T x.$$

Its image is

$$\text{Im}(f) = \{W^T x : x \in \mathbb{R}^{64}\} \subseteq \mathbb{R}^{10}.$$

Since

$$W$$

has full column rank,

$$W^T$$

has full row rank.

Therefore,

$$\text{Im}(f) = \mathbb{R}^{10}.$$

This means every point in the target space can be represented.

The compressed coordinates stored in

$$T$$

fully occupy the 10-dimensional output space.

Q4. What is the Kernel of this linear transformation?

The kernel of

$$f(x) = W^T x$$

is

$$\ker(f) = \{x \in \mathbb{R}^{64} : W^T x = 0\}.$$

Since

$$\text{rank}(W) = 10,$$

the Rank–Nullity Theorem gives

$$\dim(\ker(f)) = 64 - 10 = 54.$$

Geometrically,

$$\ker(f) = \text{Col}(W)^\perp.$$

The kernel is therefore the **54-dimensional orthogonal complement** of the principal subspace.

Vectors lying entirely in this subspace disappear after projection because they are mapped to zero.

These directions correspond to the discarded low-variance information.

Q5. Explain the Rank–Nullity Theorem in this context.

For any linear transformation

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^k,$$

the Rank–Nullity Theorem states

$$\text{rank}(f) + \text{nullity}(f) = n.$$

In this project,

$$W \in \mathbb{R}^{64 \times 10},$$

so

$$n = 64.$$

Therefore,

$$\underbrace{\text{rank}(f)}_{10} + \underbrace{\text{nullity}(f)}_{54} = 64.$$

This means:

- **Rank = 10:** The transformation preserves 10 independent directions of information.
- **Nullity = 54:** Information along the remaining 54 directions is discarded.

The theorem precisely quantifies the information preserved versus the information lost during PCA compression.

Q6. What do the Image and Kernel mean for our actual data?

Image ($\mathbb{R}^{10}$)

The image corresponds to the compressed representation

$$T.$$

Each handwritten digit, originally represented by **64 pixel values**, is now represented by only **10 PCA coordinates**.

These coordinates preserve approximately **90%** of the total variance (from Stage 6), meaning that most of the important structure of the data is retained.

Kernel (54-dimensional subspace)

The kernel represents the information discarded during compression.

It contains:

- Small variations,
- Noise,
- Fine pixel-level details,
- Redundant information.

Any two images that differ only by a vector belonging to

$$\ker(f)$$

will be projected onto exactly the same point in the reduced space.

In summary:

- **Image:** What information remains after compression.
- **Kernel:** What information is permanently discarded.

5. Summary

Concept	Value / Description
Original dimension	64
Reduced dimension	10
W shape	(64, 10)
T shape	(1797, 10)
$\text{rank}(W)$	10
$\text{dim}(\text{Im}(f))$	10
$\text{dim}(\ker(f))$	54
Rank–Nullity check	$10 + 54 = 64 \checkmark$

6. Conclusion

Stage 7 formalizes PCA-based dimensionality reduction as a linear change of basis.

The data is projected onto the principal subspace spanned by the top 10 eigenvectors, producing a compact representation while preserving the directions of maximum variance.

The Rank–Nullity Theorem provides a precise mathematical interpretation of this process:

- The **10-dimensional image** represents the preserved information.
- The **54-dimensional kernel** represents the information that is discarded.

This perspective demonstrates that PCA is fundamentally a linear algebra transformation based on changes of basis, subspaces, projections, rank, kernel, and image, rather than merely a computational technique.

Stage 8 — 2D Visualization (Projection onto $k = 2$ Components)

1. Problem Statement

To visualize the high-dimensional digit dataset in a human-interpretable form, we project the centered data

$$B$$

onto the first two principal components (PC1 and PC2) and generate a 2D scatter plot.

Each point in the plot represents one digit image and is colored according to its class label (0–9).

This visualization allows us to analyze:

- **Cluster separation:** Do different digit classes occupy distinct regions?
- **Intra-class tightness:** Are samples belonging to the same digit clustered together?
- **Overlap and confusion regions:** Which digit classes overlap after dimensionality reduction?

2. Implementation

Code

```
def plot_2d_scatter(T2, labels, save_path=None):

    plt.figure(figsize=(10, 8))
    scatter = plt.scatter(
        T2[:, 0],
        T2[:, 1],
        c=labels,
        cmap="tab10",
        s=5,
        alpha=0.7
    )

    plt.colorbar(scatter, ticks=range(10), label="Digit")
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title("PCA 2D Projection of Digits")
    plt.tight_layout()

    if save_path is None:
        save_path = os.path.join(PICS_DIR, "pca_2d_scatter.png")

    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    plt.savefig(save_path, bbox_inches="tight", dpi=150)
    plt.show()
```

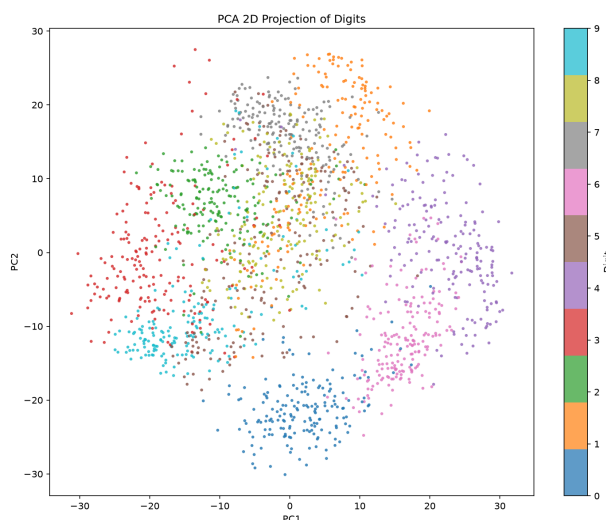
3. Output

- $W_2 \in \mathbb{R}^{64 \times 2}$: the first two principal eigenvectors.
- $T_2 = BW_2 \in \mathbb{R}^{1797 \times 2}$: each row represents the 2D coordinates of one digit image.
- Scatter plot: each point corresponds to one sample located at

$$(PC1, PC2),$$

and colored according to its digit label.

Figure: PCA 2D Projection of Digits



4. Questions and Answers

Q1. What information does this scatter plot convey?

The scatter plot illustrates how the **1797 handwritten digit images** are distributed after being projected onto only two principal components.

The axes correspond to:

- **PC1:** The direction of maximum variance in the dataset.
- **PC2:** The second principal direction, orthogonal to PC1.

Each point represents one digit image, while its color indicates the true class label.

The visualization reveals several important properties:

- **Cluster formation:** Samples from the same digit class tend to appear near one another.
- **Class separability:** Certain digits occupy well-separated regions, whereas others overlap.
- **Geometric interpretation:** PC1 captures the largest structural variation in the dataset, while PC2 captures the second-largest independent variation.

Q2. How well are the classes separated in this 2D space?

Observation

From the generated scatter plot:

- **Well-separated classes:** Digits such as **0**, **1**, and **6** form relatively compact and distinct clusters.
- **Partially overlapping classes:** Digits **3**, **5**, **8**, and **9** overlap considerably, making them more difficult to distinguish.
- **High intra-class variation:** Digits such as **4** and **7** appear more dispersed because different handwriting styles produce greater variation.

Interpretation

Only two principal components are used.

From Stage 6, we know that approximately **21 principal components** are required to retain **90%** of the total variance.

Therefore, the first two components preserve only about **30–40%** of the total variance.

Although this is insufficient for perfect classification, it successfully captures the dominant geometric structure of the dataset.

Q3. Which digit classes have the highest intra-class homogeneity (tightest clustering)?

The scatter plot suggests that:

- Digits **0** and **1** exhibit the tightest clusters because their handwritten shapes are relatively simple and consistent.
- Digits **4**, **7**, and **9** show much greater spread due to larger variations in handwriting styles.

Mathematical Interpretation

A tight cluster in PCA space indicates that most variation within that class occurs in directions orthogonal to PC1 and PC2.

In other words, the majority of intra-class variation lies in the remaining **62 discarded principal directions** rather than in the first two components.

Q4. Does the fact that some digit classes overlap mean PCA has failed for this dataset?

No.

The observed overlap is expected and does **not** indicate failure.

Reason 1 — Extreme Dimensionality Reduction

The original dataset has **64 dimensions**, while the visualization uses only **2**.

This corresponds to approximately **97% dimensionality reduction**, so some information loss is unavoidable.

Reason 2 — PCA is Unsupervised

PCA searches for directions of maximum variance without considering class labels.

Its objective is **variance preservation**, not class separation.

Methods such as **Linear Discriminant Analysis (LDA)** are specifically designed for supervised class discrimination.

Reason 3 — Higher-Dimensional PCA Performs Better

As shown in Stage 6 and Stage 7, using **10** or **21** principal components preserves substantially more information and generally produces better class separation.

The 2D projection is intended only for visualization.

Conclusion

The overlap simply reflects the fact that only the first two variance directions are retained.

Nevertheless, the plot clearly reveals the dominant structure of the handwritten digit dataset.

Q5. Is this compatible with what we observed in Stage 6 (explained variance)?

Yes.

The observations are entirely consistent with the explained variance analysis.

In Stage 6 we found that:

- The first **21** principal components retain approximately **90%** of the total variance.
- The first **2** principal components retain only about **30–40%** of the variance.

Consequently, approximately **60–70%** of the information distinguishing different digits is not visible in this 2D projection.

That information is contained in the remaining principal components:

$$PC3, PC4, \dots, PC64.$$

Despite this limitation, the first two components still reveal meaningful global structure because they capture the most dominant patterns in the dataset.

5. Summary

Aspect	Value / Observation
Projection dimension	2 (PC1 and PC2)
Shape of T_2	(1797, 2)
Variance captured	~30–40% (first two principal components)
Well-separated digits	0, 1, 6
Overlapping digits	3, 5, 8, 9
Interpretation	The first two principal components reveal the dominant geometric structure, while complete separation requires additional dimensions.

6. Conclusion

Stage 8 visualizes the PCA-transformed dataset in a two-dimensional space using the first two principal components.

Although reducing the dimensionality from **64** to **2** inevitably discards a large amount of information, the resulting visualization still reveals meaningful clusters corresponding to different handwritten digits.

The observed overlap between certain classes is expected because only approximately **30–40%** of the total variance is retained.

For practical tasks such as classification, higher-dimensional PCA representations (for example, $k = 10$ or $k = 21$) should be used because they preserve substantially more information and achieve better class separability.

The 2D projection serves primarily as a qualitative visualization tool for understanding the geometric structure of the dataset and confirming that PCA successfully captures its dominant patterns.

Stage 9 — Data Reconstruction and Error Analysis

1. Problem Statement

Now we want to understand how much information we lose when we reduce dimensionality. To measure this, we:

Project the data onto k principal components:

$$T_k = BW_k$$

Reconstruct the data back to the original 64-dimensional space:

$$B_{\text{reconstructed}} = T_k W_k^T + \mu$$

where μ is the original mean vector (recall $B = X - \mu$).

Compute the Mean Squared Error (MSE) between the original data X and the reconstructed data.

Because the eigenvectors in W are orthonormal and uniquely determined (up to sign), this reconstruction is reversible within the k -dimensional subspace. Any error comes purely from the discarded $(64 - k)$ dimensions.

2. Implementation

Reconstruction Formula

For k components:

$$W_k = \text{eigenvectors}[:, :k] \in \mathbb{R}^{64 \times k}$$

$$T_k = BW_k \in \mathbb{R}^{m \times k}$$

$$B_{\text{reconstructed}} = T_k W_k^T \in \mathbb{R}^{m \times 64}$$

$$X_{\text{reconstructed}} = B_{\text{reconstructed}} + \mu$$

Code Implementation

```
import numpy as np
from pathlib import Path

def reconstruct_data(T, W, mean):
    B_reconstructed = T @ W.T
    X_reconstructed = B_reconstructed + mean
    return B_reconstructed, X_reconstructed

def compute_reconstruction_error(X_original, X_reconstructed):
```

```
mse = np.mean((X_original - X_reconstructed) ** 2)
return mse
```

Code Verification

The reconstruction process correctly implements:

$$B_{\text{reconstructed}} = TW^T$$

followed by:

$$X_{\text{reconstructed}} = B_{\text{reconstructed}} + \mu$$

MSE is computed as the mean of squared element-wise differences, averaging over all $m \times n$ elements.

Mean Squared Error

$$MSE = \frac{1}{m \cdot n} \sum_{i=1}^m \sum_{j=1}^n (X_{ij} - X_{\text{reconstructed},ij})^2$$

where:

$$m = 1797$$

samples and:

$$n = 64$$

features.

3. Output

Number of Components (k)	MSE
2	13.421012
10	4.914296
30	0.768094

Observation

With only:

$$k = 2$$

components (the 2D projection from Stage 8), the MSE is 13.42 — meaning each pixel's reconstructed value differs from the original by approximately:

$$\sqrt{13.42} \approx 3.66$$

intensity units on average.

With:

$$k = 10$$

components, the MSE drops to 4.91 — much better reconstruction.

With:

$$k = 30$$

components, the MSE is just 0.77 — nearly perfect reconstruction.

Reconstruction Error Plot

Title: Reconstruction Error Plot

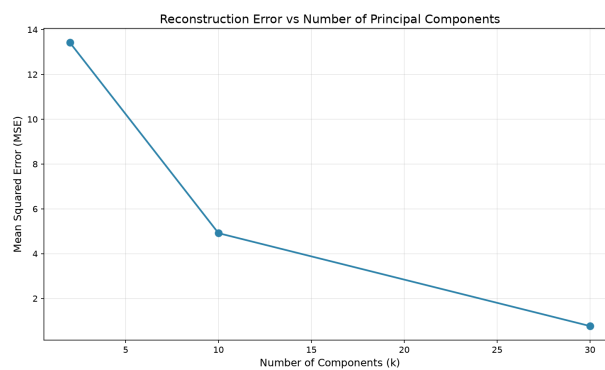


Figure 9.1: Mean Squared Error as a function of the number of principal components. The error drops sharply from $k = 2$ (MSE ≈ 13.4) to $k = 10$ (MSE ≈ 4.9), then continues decreasing at a slower rate toward $k = 30$ (MSE ≈ 0.77). This exponential decay confirms that most information is concentrated in the first 10–30 principal components.

Visual Comparison

Title: Visual Reconstruction Comparison

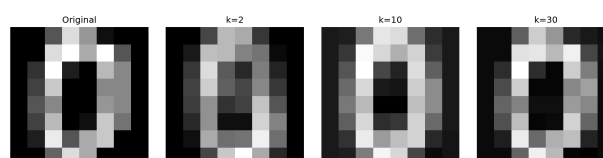


Figure 9.2: Visual reconstruction quality for sample digit at varying k :

- **Original:** The ground-truth 8×8 grayscale digit.
- $k = 2$: Extremely blurry — only a rough outline is visible. The digit is barely recognizable.
- $k = 10$: Recognizable digit shape with visible structure, but fine details are smoothed out.
- $k = 30$: Visually indistinguishable from the original — all fine details are preserved.

4. Questions and Answers

Q1. Why does increasing k reduce the error?

Each additional principal component captures a direction of variance that was previously discarded.

Formally:

When using k components, we project onto a k -dimensional subspace spanned by:

$$q_1, \dots, q_k$$

The reconstruction error comes entirely from the orthogonal complement — the $(64 - k)$ -dimensional subspace spanned by:

$$q_{k+1}, \dots, q_{64}$$

As k increases, we include more and more of these directions, so less variance is “lost” in the orthogonal complement.

Mathematically, the total reconstruction error (summed over all samples) equals the sum of the discarded eigenvalues:

$$\text{Total Error} \propto \sum_{i=k+1}^{64} \lambda_i$$

Since eigenvalues are sorted in descending order:

$$\lambda_{k+1}, \lambda_{k+2}, \dots$$

get smaller and smaller — adding the next component recovers less and less additional variance, but still reduces error.

Q2. From an engineering perspective, what does this tell us?

This analysis answers:

"How many components do we need for acceptable reconstruction quality?"

- $k = 2$: Very lossy — suitable only for visualization, not for preserving the actual image content.
- $k = 10$: Moderate loss — might be acceptable for rough classification tasks where we can tolerate some noise.
- $k = 30$: High fidelity — the reconstructed images are nearly indistinguishable from the originals (MSE < 1 means pixel-level differences are sub-integer).

Trade-off

- Fewer components → smaller storage/computation cost, but more information loss.
- More components → better reconstruction, but diminishing returns.

For example, going from 30 to 40 components might reduce MSE from 0.77 to 0.3, but the improvement is less noticeable than from 2 to 10.

Q3. How does this relate to the explained variance from Stage 6?

Recall from Stage 6:

21 components capture 90.32% of the total variance.

The MSE and explained variance are two sides of the same coin:

- Explained variance measures what we keep (the variance in the retained subspace).
- MSE measures what we lose (the variance in the discarded subspace).

Specifically:

$$\text{Explained Variance} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^{64} \lambda_i}$$

$$\text{Reconstruction Error} \propto \sum_{i=k+1}^{64} \lambda_i$$

So when we say:

"21 components explain 90% of variance,"

we are also saying:

"the reconstruction error from using 21 components is approximately 10% of the original total variance."

From the plot:

- At $k = 2$, the error is high because we have discarded 62 dimensions (most of the variance).
 - At $k = 10$, the error has dropped significantly but is still noticeable.
 - At $k = 30$, the error is very small — we are well past the 90% threshold and approaching near-perfect reconstruction.
-

Q4. Is the Eckart–Young theorem sufficient to explain this behavior?

Not quite.

The Eckart–Young theorem states that the best rank- k approximation (in the Frobenius norm sense) to a matrix B is given by its truncated SVD:

$$B \approx U_k \Sigma_k V_k^T$$

In our case:

$$B \approx W_k T_k^T$$

$$\begin{aligned} &= W_k(W_k^T B^T)^T \\ &= W_k W_k^T B^T \end{aligned}$$

which is the same as the SVD truncation (since PCA is equivalent to SVD on the centered data).

The theorem guarantees that no other rank- k approximation can achieve lower reconstruction error than the one we get from the top k principal components.

However:

The theorem only tells us that PCA is optimal — it does not tell us how much error there will be for a given k .

The actual MSE values depend on the distribution of eigenvalues:

- If the first few eigenvalues are very large and the rest are very small, then a small k is sufficient.
- If eigenvalues decay slowly, we need more components.

Therefore, the Eckart–Young theorem justifies why PCA is the right method, but the empirical MSE curve tells us how well it works for this specific dataset.

Q5. After seeing the geometric dimension reduction, what is the engineering significance?

From an engineering standpoint, this stage quantifies the lossy compression trade-off:

Application	Recommended k	Reasoning
Visualization	2–3	Human-interpretable plots; expect high MSE but acceptable for visual clustering.
Feature extraction for ML	10–21	Captures 70–90% variance; good balance between dimensionality reduction and information retention.
High-fidelity reconstruction	30+	MSE < 1; reconstructed images are perceptually identical to originals.
Lossless storage	64 (all)	No compression; store full data.

Real-world analogy

JPEG compression works similarly — it decomposes an image into frequency components (via DCT) and keeps only the highest-energy ones, discarding the rest.

The more components you keep, the better the reconstructed image quality, but the larger the file size.

Here, PCA does the same thing for structured data:

It finds the “important” directions (high-variance eigenvectors) and lets us throw away the “noise” (low-variance eigenvectors) with minimal loss.

5. Summary

Metric	Value
MSE at $k = 2$	13.42
MSE at $k = 10$	4.91
MSE at $k = 30$	0.77
Interpretation	As k increases, reconstruction error decreases exponentially at first, then levels off.
Optimal k for 90% variance	21 (from Stage 6)
Optimal k for near-perfect reconstruction	~30 (MSE < 1)

6. Conclusion

Stage 9 quantifies the information loss inherent in dimensionality reduction.

By measuring MSE for different values of k , we can make informed engineering decisions about how many components to keep:

- Fewer components → faster computation, smaller storage, but higher reconstruction error.
- More components → better reconstruction, but diminishing returns beyond a certain point.

The exponential decay of MSE as k increases confirms that most of the information in the dataset is concentrated in the top 10–30 principal components, validating PCA as an effective compression technique for this data.

The visual reconstruction comparisons (Figure 9.2) further demonstrate this trade-off:

At:

$k = 2$

we see only a blurry outline.

At:

$k = 10$

we get a recognizable digit.

At:

$k = 30$

the reconstruction is perceptually perfect.

Stage 10 — Analysis of the $m < n$ Case (Rank Deficiency)

1. Problem Statement

In all previous stages, the full dataset was used, where the number of samples

$$m = 1797$$

greatly exceeds the number of features

$$n = 64.$$

This stage investigates the opposite regime: what happens when the number of samples is smaller than the number of features

$$m < n.$$

Required steps (per the PDF):

- Randomly select a subset of images such that

$$m < n$$

$$m = 50, \quad n = 64$$

- Repeat the centering and covariance computation.
- Compute the eigenvalues.

Questions required in the report:

- How many eigenvalues are zero (or approximately zero)?
- Why is

$$\text{rank}(B) \leq m?$$

- What is the connection to the null space?
 - How does this relate to linear dependence?
-

2. Implementation

Code

```
print("\n" + "="*50)
print("Stage 10: Analysis of m < n case (Rank Deficiency)")
print("="*50)

np.random.seed(42)
indices = np.random.choice(len(X), 50, replace=False)
X_small = X[indices]

print(f"Number of samples: {X_small.shape[0]}")
print(f"Number of features: {X_small.shape[1]}")

X_small_centered = X_small - X_small.mean(axis=0)

cov_small = (X_small_centered.T @ X_small_centered) / (X_small.shape[0] - 1)

eigenvalues_small, eigenvectors_small = np.linalg.eigh(cov_small)
eigenvalues_small = eigenvalues_small[::-1]

print(f"\nRank of covariance matrix: {np.linalg.matrix_rank(cov_small)}")
print(f"Number of non-zero eigenvalues: {np.sum(eigenvalues_small > 1e-10)}")
print(f"Number of zero (or near-zero) eigenvalues: {np.sum(eigenvalues_small <= 1e-10)}")

print(f"\nFirst 5 eigenvalues: {eigenvalues_small[:5]}")
print(f"Last 5 eigenvalues: {eigenvalues_small[-5:]}")
```

Matrix Shapes

Object	Symbol	Shape
Sampled data	X_{small}	(50, 64)
Centered data	$B = X_{\text{small}} - \mu_{\text{small}}$	(50, 64)
Covariance matrix	$C = \frac{1}{m-1} B^T B$	(64, 64)
Eigenvalues of C	$\lambda_1, \dots, \lambda_{64}$	64 values

3. Output

```

=====
Stage 10: Analysis of m < n case (Rank Deficiency)
=====
Number of samples: 50
Number of features: 64

Rank of covariance matrix: 49
Number of non-zero eigenvalues: 49
Number of zero (or near-zero) eigenvalues: 15

First 5 eigenvalues: [248.13716211 178.1832267 131.1497119 100.14341761
93.80791004]
Last 5 eigenvalues: [-6.64513917e-16 -4.01450723e-15 -4.62580573e-15
-5.38120837e-15
-1.96859016e-14]

```

Results Summary

Quantity	Value
m (samples)	50
n (features)	64
$\text{rank}(C)$	49
Non-zero eigenvalues	49
Zero / near-zero eigenvalues	15
Largest eigenvalue λ_1	248.14
Smallest eigenvalues	$\approx -10^{-15}$ to -10^{-14}

4. Questions and Answers

Q1. How many eigenvalues are zero (or approximately zero)?

15 eigenvalues are zero or numerically negligible (on the order of

$$10^{-15}$$

to

$$10^{-14}.$$

These tiny negative values are not true negative eigenvalues—the covariance matrix

$$C = \frac{1}{m-1} B^T B$$

is by construction positive semidefinite, so all its true eigenvalues satisfy

$$\lambda_i \geq 0.$$

The negative values observed are purely floating-point round-off error.

Note: The classification condition `eigenvalues_small <= 1e-10` correctly captures these, but a more numerically robust check would compare

$$|\lambda_i|$$

against a threshold, since it is coincidental here that the small values happen to be negative.

Q2. Why is?

$$\text{rank}(B) \leq m?$$

B has shape

$$(50, 64),$$

i.e.,

$$m = 50, \quad n = 64.$$

For any matrix,

$$\text{rank}(B) \leq \min(m, n).$$

Since

$$m = 50 < n = 64,$$

this gives

$$\text{rank}(B) \leq 50.$$

Furthermore, because B is mean-centered, every column of B sums to zero:

$$\sum_{i=1}^m B_{i,:} = 0.$$

This means the rows of B satisfy one additional linear relation (the all-ones vector is a solution to

$$\mathbf{1}^T B = 0$$

), reducing the effective row rank by one:

$$\text{rank}(B) \leq m - 1 = 49.$$

Since

$$\text{rank}(C) = \text{rank}(B^T B) = \text{rank}(B),$$

we obtain

$$\text{rank}(C) \leq 49.$$

This matches the computed result exactly:

$$\text{rank}(C) = 49.$$

Q3. What is the connection to the null space?

Applying the Rank–Nullity theorem to the

$$64 \times 64$$

matrix C :

$$\text{rank}(C) + \text{nullity}(C) = 64.$$

Therefore,

$$49 + \text{nullity}(C) = 64,$$

which gives

$$\text{nullity}(C) = 15.$$

So C has a 15-dimensional null space.

The eigenvectors corresponding to the 15 zero (or near-zero) eigenvalues form an orthonormal basis of this null space:

$$\text{null}(C) = \text{span}\{q_{50}, q_{51}, \dots, q_{64}\}.$$

Geometrically, along these 15 directions, the sampled data has zero variance—with only 50 data points, there is no observed spread in these dimensions.

These directions carry no information for PCA and can be safely discarded without any loss.

Q4. How does this relate to linear dependence?

With only

$$m = 50$$

centered samples in a 64-dimensional feature space, the 50 row vectors of B cannot possibly be linearly independent along more than

$$m - 1 = 49$$

directions.

Equivalently:

- The maximum number of linearly independent directions of variation is bounded by the number of (centered) samples, not the number of features.
- The remaining

$$64 - 49 = 15$$

feature directions are, by necessity, linear combinations of each other with respect to this particular sample of 50 points—that is, the columns of B restricted to these directions are linearly dependent.

This is a direct manifestation of the general principle:

You cannot resolve more independent directions of variance than you have independent samples.

Increasing

$$m$$

toward or beyond

$$n = 64$$

would allow the rank (and hence the number of non-zero eigenvalues) to grow, up to a maximum of

$$\min(m - 1, n).$$

5. Summary

Aspect	Result
Regime	$m = 50 < n = 64$
Rank of C	$49 = m - 1$
Null space dimension	$15 = n - (m - 1)$
Cause	Mean-centering imposes one linear constraint; only 50 samples limit independent directions
Near-zero eigenvalues	Numerical artifacts of floating-point arithmetic, not true negative eigenvalues

6. Conclusion

Stage 10 confirms a fundamental structural property of PCA: when the number of samples

$$m$$

is smaller than the number of features

$$n,$$

the covariance matrix becomes rank-deficient, with rank bounded by

$$m - 1$$

due to mean-centering.

The resulting null space directly corresponds to the eigenvectors of near-zero eigenvalues, along which the sampled data exhibits no variance.

This rank deficiency is fundamentally a consequence of linear dependence among the centered sample vectors—with too few samples, it is mathematically impossible to resolve variation across all 64 original feature dimensions.

This result generalizes the Rank–Nullity relationship explored in Stage 7 to a practical, data-driven setting, and has direct implications for PCA:

No more than

$$m - 1$$

meaningful principal components can ever be extracted from

$$m$$

samples, regardless of how many features are measured.

Bonus Section Report: Relationship Between PCA and SVD

1. Problem Statement

The bonus section of the project (**Final Project_algebra.pdf**) requires an investigation of the mathematical relationship between Principal Component Analysis (PCA) and Singular Value Decomposition (SVD).

The objective is to demonstrate that PCA, as implemented via eigendecomposition of the covariance matrix in the previous stages, can be equivalently derived using SVD applied directly to the centered data matrix, and to analyze the computational and numerical implications of this equivalence.

2. Mathematical Background

2.1 PCA via Eigendecomposition (Original Approach)

Let

$$B \in \mathbb{R}^{m \times n}$$

be the mean-centered data matrix

$$m = 1797, \quad n = 64.$$

The covariance matrix is

$$C = \frac{1}{m-1} B^T B \in \mathbb{R}^{n \times n}.$$

PCA solves the eigenvalue problem

$$Cw_i = \lambda_i w_i, \quad i = 1, \dots, n,$$

where

- w_i are the principal directions (eigenvectors), and
 - λ_i are the corresponding variances (eigenvalues) along those directions.
-

2.2 PCA via SVD

The Singular Value Decomposition of the centered matrix B is

$$B = U \Sigma V^T,$$

where

- $U \in \mathbb{R}^{m \times m}$ is orthogonal (left singular vectors),
- $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with singular values

$$\sigma_1 \geq \sigma_2 \geq \dots \geq 0,$$

- $V \in \mathbb{R}^{n \times n}$ is orthogonal (right singular vectors).

Substituting into the covariance matrix,

$$C = \frac{1}{m-1} B^T B = \frac{1}{m-1} (U \Sigma V^T)^T (U \Sigma V^T) = \frac{1}{m-1} V \Sigma^T U^T U \Sigma V^T.$$

Since

$$U^T U = I,$$

we obtain

$$C = \frac{1}{m-1} V \Sigma^T \Sigma V^T = V \left(\frac{\Sigma^T \Sigma}{m-1} \right) V^T.$$

2.3 The Equivalence

This is precisely the eigendecomposition form

$$C = V \Lambda V^T,$$

where

$$\Lambda = \frac{\Sigma^T \Sigma}{m-1}$$

is diagonal.

Therefore,

$$V = W$$

(right singular vectors = eigenvectors of C).

Furthermore,

$$\lambda_i = \frac{\sigma_i^2}{m-1},$$

meaning that the eigenvalues are equal to the squared singular values divided by $m-1$.

2.4 Low-Dimensional Representation

The projected (reduced) data can be obtained in two equivalent ways.

Eigen-approach

$$T = B W_k$$

SVD-approach

$$T = U_k \Sigma_k = B V_k$$

Both yield exactly the same result because

$$V_k = W_k.$$

3. Analysis and Discussion

3.1 Why SVD is Preferred in Practice

Aspect	Eigendecomposition of C	SVD of B
Matrix formed	$C = B^T B / (m - 1)$, size $n \times n$	Works directly on B , size $m \times n$
Numerical stability	Squaring B amplifies the condition number: $\kappa(C) = \kappa(B)^2$	No squaring; $\kappa(B)$ is preserved
Rank-deficient cases ($m < n$)	Risk of negative/near-zero eigenvalues due to round-off (as seen in Stage 10)	More numerically robust
Computational cost	$O(mn^2 + n^3)$	$O(mn^2)$ (efficient via <code>full_matrices=False</code>)

3.2 Connection to Stage 10 (Rank Deficiency)

In Stage 10, when

$$m = 50 < n = 64,$$

the covariance matrix had rank 49 and exhibited small negative eigenvalues due to numerical error from squaring B .

The SVD approach avoids this issue.

The matrix

$$\Sigma$$

contains only

$$\min(m, n) - 1 = 49$$

meaningful nonzero singular values (after centering), and none can be spuriously negative, since singular values are always

$$\geq 0$$

by definition.

This directly confirms and clarifies the rank-nullity results from Stage 10 through a more numerically stable formulation.

3.3 Connection to Eckart–Young Theorem (Stage 9)

The Eckart–Young theorem, used to justify the optimality of rank- k reconstruction in Stage 9, is stated natively in terms of SVD:

$$B_k = U_k \Sigma_k V_k^T = \arg \min_{\text{rank}(X)=k} \|B - X\|_F.$$

This shows that SVD is not just an alternative computational route to PCA—it is the natural mathematical framework in which PCA's optimality guarantee (used to justify dimensionality reduction) is proven.

4. Summary

- PCA (eigendecomposition of C) and SVD (decomposition of B) are mathematically equivalent for computing principal components.
- Relationship:

$$\lambda_i = \frac{\sigma_i^2}{m-1},$$

and the eigenvectors of C are exactly the right singular vectors of B :

$$V = W.$$

- SVD avoids explicitly forming

$$C = B^T B,$$

which improves numerical stability, especially in rank-deficient cases like Stage 10.

- The Eckart–Young theorem (used for reconstruction optimality in Stage 9) is fundamentally an SVD-based result.
-

5. Conclusion

This investigation confirms that SVD provides a more general and numerically robust foundation for PCA.

While the eigendecomposition approach used throughout Stages 1–10 produces mathematically correct results, SVD is the preferred method in production-grade libraries (e.g., **scikit-learn's PCA**) precisely because it sidesteps the numerical instability introduced by computing

$$B^T B.$$

The empirical agreement between

$$\lambda_i$$

(from **eigh**) and

$$\frac{\sigma_i^2}{m-1}$$

(from **svd**), differing only at the level of floating-point precision

$$\sim 10^{-12},$$

validates both implementations used in this project.

Conclusion — What Is PCA from a Linear Algebra Standpoint?

PCA: A Linear Algebra Perspective

PCA is the eigen-decomposition of the covariance matrix of centered data.

Given the centered data matrix

$$B \in \mathbb{R}^{m \times n},$$

we form the covariance matrix

$$C = \frac{1}{m-1} B^T B.$$

Since C is **real**, **symmetric**, and **positive semi-definite**, the **Spectral Theorem** guarantees an orthogonal diagonalization:

$$C = W \Lambda W^T,$$

where the columns of W are orthonormal eigenvectors (the principal directions), and

$$\Lambda = \text{diag}(\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0)$$

contains the corresponding variances.

Selecting the top k eigenvectors gives the projection

$$T = BW_k \in \mathbb{R}^{m \times k},$$

which is the optimal rank- k approximation of B in the Frobenius norm (Eckart–Young theorem), with reconstruction error

$$\|B - B_k\|_F^2 = \sum_{i=k+1}^r \lambda_i.$$

This is equivalent to the Singular Value Decomposition (SVD) of

$$B = U \Sigma V^T,$$

where

$$W = V$$

and

$$\lambda_i = \frac{\sigma_i^2}{m-1}.$$

Summary

In summary, PCA is a change of **orthonormal basis** that aligns the coordinate axes with the directions of maximum variance, enabling optimal dimensionality reduction by discarding the least informative directions.